

Dokumentacja Biblioteki PUTM EV CAN

Michał Błotniak

28 stycznia 2026

Streszczenie

Niniejszy dokument stanowi instrukcję obsługi biblioteki `PUTM_EV_CAN_LIBRARY`. Biblioteka została zaprojektowana dla systemów wbudowanych (STM32) oraz środowiska ROS2, kładąc nacisk na statyczną alokację pamięci (Zero-Allocation), bezpieczeństwo typów (C++20) oraz automatyczną obsługę ramek DBC. Dokument definiuje również obowiązującą w zespole konwencję nazewnictwa magistral CAN oraz opisuje funkcje diagnostyczne.

Spis treści

1	Integracja i Konfiguracja w CMake	2
1.1	Dodawanie biblioteki do projektu	2
1.2	Wybór konfiguracji (STM32 vs ROS2)	2
1.2.1	Konfiguracja dla STM32	2
1.2.2	Konfiguracja dla ROS2 / Linux	2
1.3	Włączenie obsługi CAN FD	2
2	Konwencja Nomenklatury Magistral	3
3	Poprawna inicjalizacja	3
3.1	Krok 1: Instancje Globalne	3
3.2	Krok 2: Wywołanie Init	3
4	Diagnostyka LED	4
4.1	Konfiguracja Diody	4
4.2	Znaczenie Sygnałów (Mruganie)	4
5	Wysyłanie danych (TX)	5
6	Odbieranie danych (RX)	6
6.1	Krok 1: Rejestracja Callbacku	6
6.2	Krok 2: Polling w pętli głównej	6
7	Najczęściej popełniane błędy	6
8	Szczegółowa Architektura Systemu	7
8.1	Warstwa 1: Fasada Aplikacji (<code>app_can_lib</code>)	7
8.2	Warstwa 2: Logika i Routing (<code>can_interface</code> , <code>message_handler</code>)	7
8.2.1	Moduł Interfejsu (<code>can_interface</code>)	7
8.2.2	Moduł Routera Wiadomości (<code>message_handler</code>)	7
8.3	Warstwa 3: Abstrakcja Sprzętowa HAL (<code>can_hal</code>)	8
8.4	Warstwa 4: Definicje Danych (<code>dbc/generated</code>)	8

1 Integracja i Konfiguracja w CMake

Biblioteka jest dystrybuowana jako moduł CMake. Aby użyć jej w projekcie, należy zmodyfikować główny plik `CMakeLists.txt` projektu.

1.1 Dodawanie biblioteki do projektu

W głównym pliku `CMakeLists.txt` (np. w katalogu głównym projektu STM32 lub węzła ROS2) należy dodać podkatalog z biblioteką oraz zlinkować ją z plikiem wykonywalnym.

```
1 # 1. Dodanie podkatalogu biblioteki
2 add_subdirectory(Core/Inc/PUTM_EV_CAN_LIBRARY)
3
4 # 2. Linkowanie biblioteki z twoim projektem
5 # 'twoj_projekt' to nazwa zdefiniowana w add_executable()
6 target_link_libraries(twoj_projekt PRIVATE putm_ev_can)
```

Listing 1: Podstawowa konfiguracja CMake

1.2 Wybór konfiguracji (STM32 vs ROS2)

Biblioteka wspiera dwa backhendy sprzętowe. Wybór następuje poprzez ustawienie zmiennej `PUTM_BACKEND` przed wywołaniem `add_subdirectory`.

1.2.1 Konfiguracja dla STM32

Jest to domyślna konfiguracja. Wymaga ona, aby w projekcie dostępna była biblioteka HAL.

```
1 set(PUTM_BACKEND "STM32" CACHE STRING "Select backend" FORCE)
2 add_subdirectory(Core/Inc/PUTM_EV_CAN_LIBRARY)
```

Listing 2: Konfiguracja dla STM32

1.2.2 Konfiguracja dla ROS2 / Linux

W tym trybie biblioteka korzysta z systemowego SocketCAN.

```
1 set(PUTM_BACKEND "LINUX" CACHE STRING "Select backend" FORCE)
2 add_subdirectory(lib/PUTM_EV_CAN_LIBRARY)
```

Listing 3: Konfiguracja dla Linux/ROS2

1.3 Włączenie obsługi CAN FD

Domyślnie biblioteka pracuje w trybie Classic CAN. Aby włączyć obsługę ramek do 64 bajtów:

```
1 option(PUTM_USE_CAN_FD "Enable CAN FD" ON)
```

2 Konwencja Nomenklatury Magistral

W projekcie pojazdu zdefiniowano trzy fizyczne magistrale CAN. Aby zachować spójność w kodzie (na STM32 i w ROS2), **należy bezwzględnie** stosować poniższe nazewnictwo instancji klasy AppCAN.

Nazwa Instancji	Funkcja Magistrali	Opis
can_m	Main	Główna magistrala, większość danych, telemetria.
can_pt	Powertrain (ECU/INV)	Komunikacja wyłącznie falownik - silniki.
can_dv	Driverless	Wymiana sygnałów w trybie jazdy autonomicznej.

Tabela 1: Obowiązująca nomenklatura instancji AppCAN

Wszystkie przykłady w dalszej części dokumentacji będą odwoływać się do tych konkretnych nazw zmiennych.

3 Poprawna inicjalizacja

Biblioteka nie używa dynamicznej alokacji pamięci. Instancje muszą być zadeklarowane jako zmienne globalne.

3.1 Krok 1: Instancje Globalne

W pliku main.cpp (lub odpowiedniku w ROS2) należy utworzyć obiekty dla wykorzystywanych magistral.

```
1 /* Private variables */
2 // Definicja instancji zgodnie z przyjętą nomenklaturą
3 AppCAN can_m;    // Main Bus
4 AppCAN can_pt;   // Powertrain Bus
5 AppCAN can_dv;   // Driverless Bus
```

3.2 Krok 2: Wywołanie Init

Inicjalizację należy przeprowadzić wewnątrz funkcji main(), po konfiguracji peryferiów przez CubeMX. Każdej instancji przypisujemy odpowiedni uchwyt sprzętowy (hfdcanX).

```
1 /* USER CODE BEGIN 2 */
2
3 // 1. Inicjalizacja Main CAN (zazwyczaj FDCAN1)
4 if (!can_m.Init(&hfdcan1)) {
5     Error_Handler(); // Błąd inicjalizacji Main
6 }
7
8 // 2. Inicjalizacja Powertrain CAN (zazwyczaj FDCAN2)
9 if (!can_pt.Init(&hfdcan2)) {
10    Error_Handler(); // Błąd inicjalizacji PT
11 }
12
13 // 3. Inicjalizacja Driverless CAN (zazwyczaj FDCAN3)
14 if (!can_dv.Init(&hfdcan3)) {
15    Error_Handler(); // Błąd inicjalizacji DV
16 }
17
18 /* USER CODE END 2 */
```

Uwaga: Należy zweryfikować w konfiguracji CubeMX, który hfdcanX odpowiada fizycznie której magistrali na PCB.

4 Diagnostyka LED

Biblioteka posiada wbudowany mechanizm automatycznej sygnalizacji stanu magistrali za pomocą diody LED. Funkcjonalność ta pozwala na szybką diagnostykę problemów (np. odpięta wtyczka, uszkodzona magistrala, błędy ramek) bez konieczności podłączania debuggera.

4.1 Konfiguracja Diody

Dla każdej instancji AppCAN (np. `can_m`, `can_pt`) można przypisać osobną diodę. Konfigurację należy wykonać **raz**, zaraz po pomyślnej inicjalizacji biblioteki.

```
1 // Przykład dla Main CAN
2 if (can_m.Init(&hfdcan1)) {
3     // Przypisanie diody (np. LED1) do tej magistrali
4     can_m.ConfigStatusLed(LED1_GPIO_Port, LED1_Pin);
5 }
6
7 // Przykład dla Powertrain CAN
8 if (can_pt.Init(&hfdcan2)) {
9     // Przypisanie innej diody (np. LED2) do tej magistrali
10    can_pt.ConfigStatusLed(LED2_GPIO_Port, LED2_Pin);
11 }
```

4.2 Znaczenie Sygnałów (Mruganie)

Po skonfigurowaniu, biblioteka automatycznie steruje diodą wewnątrz metody `Poll()`. Użytkownik nie musi (i nie powinien) ręcznie sterować tymi pinami.

Zachowanie Diody	Stan Magistrali	Opis
Mruganie 1 Hz (500ms ON, 500ms OFF)	OK	Magistrala działa poprawnie (Error Active). Komunikacja jest możliwa.
Nierówne Mruganie (Podwójny błysk co 1s)	WARNING	Wykryto błędy transmisji (Error Passive / Warning). Może oznaczać zakłócenia lub błędy ramek.
Świeci Ciągłe (Solid ON)	BUS OFF	Krytyczny błąd magistrali. Kontroler CAN odłączył się od sieci (np. zwarcie).

Tabela 2: Kody diagnostyczne diody statusowej

5 Wysłanie danych (TX)

Wysyłanie odbywa się poprzez metodę `Send` wywołaną na odpowiedniej instancji magistrali (zazwyczaj `can_m`).

```
1 // Przykład w petli głównej (co 100ms)
2 if (HAL_GetTick() - last_tx > 100) {
3     last_tx = HAL_GetTick();
4
5     // 1. Tworzenie i zerowanie struktury
6     PUTM_CAN_1_pc_main_data_t tx_msg = {0};
7
8     // 2. Wypełnianie danych
9     tx_msg.vehicle_speed = 125;
10    tx_msg.inverters_ready = 1;
11
12    // 3. Wysłanie na magistralę MAIN (can_m)
13    if (can_m.Send(PUTM_CAN_1_PC_MAIN_DATA_FRAME_ID, tx_msg)) {
14        // Opcjonalnie: można tu dodać inną logikę,
15        // ale dioda statusowa jest obsługiwana automatycznie.
16    }
17 }
```

Jeśli dane (np. sterowanie falownikiem) mają trafić na magistralę Powertrain, używamy obiektu `can_pt`:

```
1 can_pt.Send(PUTM_CAN_1_AMK_SETPPOINTS_FRAME_ID, amk_msg);
```

6 Odbieranie danych (RX)

Odbiór wymaga rejestracji callbacków dla konkretnych instancji magistral oraz cyklicznego odpytywania (polling).

6.1 Krok 1: Rejestracja Callbacku

Rejestrację wykonujemy raz, przed pętlą główną. Należy zdecydować, z której magistrali spodziewamy się danej ramki.

```
1 // Odbiór danych z Main CAN (np. input od kierowcy)
2 can_m.RegisterCallback<PUTM_CAN_1_driver_input_t>(  
3     PUTM_CAN_1_DRIVER_INPUT_FRAME_ID,  
4     [](const PUTM_CAN_1_driver_input_t& msg) {  
5         // Logika obsługi (wywoływana automatycznie)  
6         if (msg.pedal_position > 1000) {  
7             // ...  
8         }  
9     }  
10 );  
11  
12 // Odbiór danych z Powertrain CAN (np. status falownika)  
13 can_pt.RegisterCallback<PUTM_CAN_1_amk_actual_values_1_t>(  
14     PUTM_CAN_1_AMK_ACTUAL_VALUES_1_FRAME_ID,  
15     [](const PUTM_CAN_1_amk_actual_values_1_t& msg) {  
16         // ...  
17     }  
18 );
```

6.2 Krok 2: Polling w pętli głównej

W pętli nieskończonej należy obsługiwać **wszystkie** aktywne instancje. Wywołanie Poll() obsługuje zarówno odbiór wiadomości, jak i sterowanie diodą diagnostyczną.

```
1 while (1) {  
2     // Obsługa buforów i diod wszystkich magistral  
3     can_m.Poll();  
4     can_pt.Poll();  
5     can_dv.Poll();  
6  
7     // ... reszta kodu ...  
8 }
```

7 Najczęściej popełniane błędy

- **Pomylenie instancji:** Wysłanie wiadomości na can_m zamiast can_pt, przez co falownik jej nie otrzyma (separacja fizyczna sieci).
- **Brak wywołania Poll() dla jednej z instancji:** Skutkuje przepełnieniem bufora sprzętowego na tej magistrali, brakiem wywołań callbacków oraz brakiem sygnalizacji LED.
- **Zła konfiguracja zegarów FDCAN:** Jeśli Init() zwraca false, sprawdź Clock Configuration w CubeMX.
- **Brak inicjalizacji struktury zerami:** Zawsze używaj Typ zmienna = {0};.

8 Szczegółowa Architektura Systemu

Biblioteka została zaprojektowana w oparciu o wzorzec projektowy *Fasada* (Facade) oraz *Wstrzykiwanie Zależności* (Dependency Injection via Composition). Architektura jest podzielona na cztery główne warstwy, co zapewnia separację logiki biznesowej od sprzętu.

8.1 Warstwa 1: Fasada Aplikacji (app_can_lib)

Jest to jedyna warstwa, z którą bezpośrednio intereaguje użytkownik końcowy.

- **Pliki:** `app_can_lib.hpp`, `app_can_lib.cpp`
- **Klasa:** `AppCAN`
- **Odpowiedzialność:**
 - Ukrywa złożoność wewnętrzną biblioteki (użytkownik widzi tylko metody `Init`, `Send`, `RegisterCallback`)
 - Zarządza czasem życia obiektów niższych warstw (`HAL`, `Interface`, `Handler`).
 - Obsługuje automatyczną diagnostykę LED w metodzie `Poll()`.
 - Nie wykonuje alokacji dynamicznej (wszystkie komponenty są składowymi klasy).

8.2 Warstwa 2: Logika i Routing (can_interface, message_handler)

Warstwa ta odpowiada za tłumaczenie surowych bajtów CAN na struktury C++ oraz kierowanie wiadomości do odpowiednich funkcji.

8.2.1 Moduł Interfejsu (can_interface)

- **Pliki:** `can_interface.hpp`, `can_interface.cpp`
- **Odpowiedzialność:**
 - Zawiera `DBC_REGISTRY` – statyczną tablicę mapującą ID ramek na funkcje pakujące/rozpakowujące.
 - Metoda `send<MsgType>()` automatycznie wyszukuje ID ramki na podstawie typu struktury, serializuje dane do bufora 8-bajtowego i przekazuje do `HAL`.
 - Metoda `process_received_messages()` pobiera ramki z `HAL` i przekazuje je do `MessageHandler`.

8.2.2 Moduł Routera Wiadomości (message_handler)

- **Pliki:** `message_handler.hpp`, `message_handler.cpp`
- **Odpowiedzialność:**
 - Przechowuje tablicę callbacków (`std::array`) zarejestrowanych przez użytkownika.
 - Zapewnia bezpieczeństwo wątkowe (Thread Safety) poprzez mechanizm `ScopedLock` (wyłączanie przerwań na czas modyfikacji tablicy callbacków).
 - Gdy przychodzi ramka o danym ID, dekoduje ją (używając funkcji z `DBC`) i wywołuje funkcję lambda użytkownika.

8.3 Warstwa 3: Abstrakcja Sprzętowa HAL (can_hal)

Warstwa izolująca kod biblioteki od konkretnego mikrokontrolera.

- **Pliki:** `can_hal.hpp` (Interfejs), `can_hal_stm32.hpp/.cpp` (Implementacja)
- **Klasa:** `ICanHal` (bazowa), `Stm32CanHal` (pochodna)
- **Odpowiedzialność:**
 - **Izolacja:** Dzięki klasie wirtualnej `ICanHal`, reszta biblioteki nie wie, że działa na STM32. Umożliwia to łatwe portowanie na Linux/ROS2.
 - **Obsługa rejestrów:** Klasa `Stm32CanHal` bezpośrednio wywołuje funkcje biblioteki producenta (np. `HAL_FDCAN_AddMessageToTxFifoQ`).
 - **Diagnostyka:** Odczytuje rejestry błędów (Error Passive, Bus Off) i zwraca zunifikowany status `BusStatus`.

8.4 Warstwa 4: Definicje Danych (dbc/generated)

Kod generowany automatycznie przez skrypt Python na podstawie plików `.dbc`.

- **Pliki:** `PUTM_CAN_M.h` (i inne zależne od nazwy pliku DBC)
- **Odpowiedzialność:**
 - Definiuje struktury danych (np. `PUTM_CAN_M_bms_lv_main_t`).
 - Definiuje stałe (ID ramek, długości DLC, współczynniki skalowania).
 - Zawiera funkcje `pack` i `unpack`, które zamieniają sygnały fizyczne (float, int) na surowe bajty ramki CAN.
 - Pliki te nie zawierają logiki sterowania, jedynie definicje danych.